

Pain Point Analysis

Review capacity does not scale with organizational size

With approximately 700 engineers submitting change requests through the same fixed review chain (Team Lead → CCB → Quality), every request is processed by the same small group of reviewers, regardless of how many engineers are generating requests. As the organization grows, request volume increases, but reviewer capacity stays fixed creating a structural bottleneck rather than an occasional delay.

No risk-based prioritization

The process applies identical scrutiny to every change, regardless of its actual complexity or risk. A minor labeling fix goes through the same Team Lead → CCB → Quality chain as a safety-critical structural redesign. This has two consequences: reviewer time is spent on low-risk changes that could reasonably be approved with lighter oversight, and — more critically — high-priority changes queue behind low-priority ones with no mechanism to fast-track them, since the process treats all requests as equally urgent.

No visibility into status or timelines

The process provides no mechanism for the requesting engineer to track where their request currently stands or how long it is likely to take. Once submitted, a request effectively disappears from the requester's view until a decision is made — there is no status update, no estimated timeline, and no visibility into whether a request is progressing normally or stuck at a particular stage. This makes it difficult for engineers to plan dependent work, and it means delays can go unnoticed internally, since nothing in the process flags a request that has been sitting too long at any one stage.

No cross-regional or cross-product-line visibility

The process handles each change request in isolation, with no shared view across product lines or regions. If two engineers in different parts of the organization submit similar or conflicting changes around the same time, nothing in the process surfaces this overlap — each request moves through its own independent review chain unaware of the other. This creates a risk of duplicated effort (two teams solving the same problem separately) or, more seriously, conflicting changes both being approved without anyone catching the conflict before implementation.